

Лабораторная работа №4

Криптосистема Мак-Элиса на двоичных кодах Гоппы

Пояснительная записка к защите. Реализация: `lab4/src`, язык `Rust`

Содержание

1. Что сделано в работе	3
2. Обозначения	3
3. Конечные поля	4
3.1. Поле $\text{GF}(2)$	4
3.2. Поле $\text{GF}(2^m)$	4
3.3. Примитивный элемент и таблицы	4
3.4. Квадрат и квадратный корень	4
4. Многочлены над $\text{GF}(2^m)$	4
4.1. Кольцо, деление, НОД	4
4.2. Фактор-кольцо $\text{GF}(2^m)[x]/(g)$	5
4.3. Квадрат и корень по модулю g	5
4.4. Неприводимость: тест Бен-Ора	5
5. Линейные коды	6
5.1. Определения	6
5.2. Расстояние и исправление ошибок	6
5.3. Синдром и трудность декодирования	6
6. Двоичные коды Гоппы	6
6.1. Определение	6
6.2. От определения к проверочной матрице	6
6.3. Параметры	7
6.4. Построение G	7
7. Декодер Паттерсона	7
7.1. Постановка	7
7.2. Ключевое уравнение	7
7.3. Разложение на квадраты	8
7.4. Решение укороченным алгоритмом Евклида	8
7.5. Поиск корней и проверки	8
8. Криптосистема	9
8.1. Генерация ключей	9
8.2. Зашифрование и расшифрование	9
8.3. Корректность	9
8.4. Зачем нужны S и P	9
9. Кодирование сообщений	9
10. Стойкость	10
10.1. Две атаки	10
10.2. Декодирование по информационным множествам	10
10.3. Слабости базовой схемы	10
10.4. Современные параметры	11
11. Сквозной пример на $\text{GF}(16)$	11

11.1. Одна ошибка (в позиции α^3)	11
11.2. Две ошибки (в позициях α^3 и α^{10})	11
12. Реализация	12
12.1. Где что лежит	12
12.2. Что проверяют тесты	12
13. Вопросы на защите	13

1. Что сделано в работе

Реализована асимметричная криптосистема Мак-Элиса (1978) на двоичных кодах Гоппы с декодером Паттерсона. Вся алгебра написана с нуля, без сторонних библиотек: конечные поля $GF(2^m)$, многочлены над ними, матрицы над $GF(2)$, построение кода и его декодирование. Из внешних зависимостей только `rand`.

Модуль	Что реализует	Раздел
<code>encryption/gf.rs</code>	поле $GF(2^m)$, $m = 2..16$: таблицы степеней и логарифмов, a^{-1} , $\sqrt{a}$	3
<code>encryption/poly.rs</code>	многочлены над $GF(2^m)$: деление, НОД, обращение и корень по модулю, тест Бен-Ора	4
<code>encryption/matrix.rs</code>	битовые векторы, матрицы и перестановки над $GF(2)$, метод Гаусса	5
<code>encryption/goppa.rs</code>	код Гоппы: матрицы H и G , синдром, декодер Паттерсона	6, 7
<code>encryption/mceliece.rs</code>	ключи $G' = SGP$, шифрование блоков и сообщений	8, 9

Проверяется командой `cargo test -p lab4`: 51 тест, от таблиц умножения до параметров статьи 1978 года ($n = 1024$, $t = 50$).

2. Обозначения

Символ	Смысл	1978 г.
m	степень расширения поля $GF(2^m)$	10
n	длина кода, $n \leq 2^m$	1024
t	число исправляемых ошибок, степень многочлена Гоппы	50
k	размерность кода: бит сообщения в блоке, $k \geq n - mt$	524
$L = (\alpha_1, \dots, \alpha_n)$	носитель: n различных элементов $GF(2^m)$	секрет
$g(x)$	неприводимый многочлен Гоппы степени t над $GF(2^m)$	секрет
G, H	порождающая ($k \times n$) и проверочная ($mt \times n$) матрицы кода	секрет
S	невырожденная матрица $k \times k$ над $GF(2)$	секрет
P	перестановка n позиций	секрет
$G' = SGP$	открытая матрица, $k \times n$	открыт
e	вектор ошибок длины n ровно с t единицами	случаен
$w(v)$	вес Хэмминга: число ненулевых координат	
$d(u, v)$	расстояние Хэмминга: $w(u - v)$	

Соглашения

- Векторы – строки, матрица действует справа: $c = mG$.
- В характеристике 2 сложение совпадает с вычитанием и равно XOR, поэтому $x - \alpha$ и $x + \alpha$ – один и тот же многочлен, а $-1 = 1$.
- Переменная z живёт **внутри** элементов поля, переменная x – в многочленах **над** полем. Коэффициенты многочлена Гоппы – элементы поля, то есть сами многочлены от z ; смешивать их нельзя.

3. Конечные поля

3.1. Поле GF(2)

Два элемента $\{0, 1\}$, арифметика по модулю 2: сложение – XOR, умножение – AND. Это единственное поле, где $1 + 1 = 0$ и потому $a + a = 0$ для любого элемента. Все матрицы и векторы криптосистемы живут над ним, и линейная алгебра над GF(2) сводится к битовым операциям над машинными словами.

3.2. Поле GF(2^m)

Конечное поле из q элементов существует тогда и только тогда, когда $q = p^m$ для простого p , и единственно с точностью до изоморфизма. При $m > 1$ оно строится не арифметикой по модулю 2^m (там $2 \cdot 2^{m-1} = 0$ – делители нуля), а как фактор-кольцо многочленов:

$$\text{GF}(2^m) = \text{GF}(2)[z]/(f(z)), \quad \deg f = m, \quad f \text{ неприводим.}$$

Элемент – многочлен от z степени меньше m с битовыми коэффициентами, то есть m -битное число. Сложение – XOR. Умножение – произведение многочленов с остатком от деления на f . Неприводимость f – аналог простоты модуля: она гарантирует, что у каждого ненулевого элемента есть обратный.

3.3. Примитивный элемент и таблицы

Мультипликативная группа $\text{GF}(2^m)^*$ циклическая порядка $2^m - 1$. Если f **примитивен**, то его корень $\alpha = z$ порождает её: $\text{GF}(2^m)^* = \{\alpha^0, \alpha^1, \dots, \alpha^{2^m-2}\}$. Тогда любой ненулевой элемент однозначно записывается как α^i , и

$$\alpha^i \cdot \alpha^j = \alpha^{(i+j) \bmod (2^m-1)}, \quad (\alpha^i)^{-1} = \alpha^{2^m-1-i}.$$

Программа строит таблицы $\exp[i] = \alpha^i$ и $\log[\alpha^i] = i$, и умножение становится сложением индексов. Строятся они многократным умножением на α – сдвигом влево и XOR с f , когда вылез бит z^m . Примитивность при этом проверяется сама: степени α обязаны пробежать все $2^m - 1$ ненулевых элементов, не повторившись и не обнулившись. Для каждого m в программе зашит свой примитивный многочлен.

Поле GF(16) с $f = z^4 + z + 1$ (оно используется в сквозном примере):

i	α^i	i	α^i	i	α^i	i	α^i
0	0001	4	0011	8	0101	12	1111
1	0010	5	0110	9	1010	13	1101
2	0100	6	1100	10	0111	14	1001
3	1000	7	1011	11	1110		

Например, $\alpha^4 = z^4 = z + 1$, потому что $z^4 + z + 1 = 0$, и $\alpha^7 = z^3 + z + 1$. Многочлен $z^4 + z^3 + z^2 + z + 1$ тоже неприводим, но не примитивен: его корень имеет порядок 5, так как $z^5 - 1 = (z - 1)(z^4 + z^3 + z^2 + z + 1)$. Программа такой многочлен отвергает.

3.4. Квадрат и квадратный корень

В характеристике 2 возведение в квадрат аддитивно: $(a + b)^2 = a^2 + 2ab + b^2 = a^2 + b^2$. Отображение Фробениуса $a \mapsto a^2$ – автоморфизм поля, поэтому корень существует у **каждого** элемента и единствен. Так как $a^{2^m} = a$,

$$\sqrt{a} = a^{2^{m-1}}, \quad \text{т. е.} \quad \sqrt{\alpha^i} = \alpha^{i \cdot 2^{m-1} \bmod (2^m-1)}.$$

Проверка: $(a^{2^{m-1}})^2 = a^{2^m} = a$.

4. Многочлены над GF(2^m)

4.1. Кольцо, деление, НОД

$\text{GF}(2^m)[x]$ – многочлены от x с коэффициентами из поля. Как и для целых чисел, есть деление с остатком: для $b \neq 0$ найдутся единственные q, r с $a = qb + r$, $\deg r < \deg b$. Деление в столбик на каждом шаге делит старший коэффициент на старший коэффициент b – здесь нужно `inv` в поле.

Расширенный алгоритм Евклида. Для (g, b) строятся последовательности

$$\begin{aligned} r_{-1} &= g, & r_0 &= b, & v_{-1} &= 0, & v_0 &= 1, \\ r_{i+1} &= r_{i-1} - q_i r_i, & v_{i+1} &= v_{i-1} - q_i v_i, \end{aligned}$$

где q_i – частное от деления r_{i-1} на r_i . Индукцией проверяется инвариант

$$r_i \equiv v_i \cdot b \pmod{g}, \quad \deg v_i = \deg g - \deg r_{i-1}.$$

Последний ненулевой остаток – НОД. Если он константа c , то $b^{-1} \equiv c^{-1} v_i \pmod{g}$ – так вычисляется `inv_mod`.

4.2. Фактор-кольцо $\text{GF}(2^m)[x]/(g)$

Остатки по модулю g степени t образуют кольцо из $(2^m)^t = 2^{mt}$ элементов. Оно является **полем** тогда и только тогда, когда g неприводим (ровно как $\mathbb{Z}/p$ для простого p). В этом поле работает декодер Паттерсона, и отсюда главное требование к многочлену Гоппы: без неприводимости не будет ни обратных, ни корней.

4.3. Квадрат и корень по модулю g

Фробениус работает и для многочленов:

$$\left(\sum_i c_i x^i \right)^2 = \sum_i c_i^2 x^{2i},$$

поперёк члены $2c_i c_j x^{i+j}$ исчезают. Отсюда любой многочлен раскладывается на чётную и нечётную части:

$$p(x) = p_0(x)^2 + x \cdot p_1(x)^2, \quad p_0 = \sum_i \sqrt{c_{2i}} x^i, \quad p_1 = \sum_i \sqrt{c_{2i+1}} x^i,$$

и потому

$$\sqrt{p} = p_0 + \sqrt{x} \cdot p_1 \pmod{g}.$$

Нужен только $\sqrt{x} \pmod{g}$. Поле $\text{GF}(2^m)[x]/(g)$ содержит 2^{mt} элементов, поэтому, как и в разделе 3.4, $\sqrt{x} = x^{2^{mt-1}}$ – это $mt - 1$ возведений в квадрат по модулю g . Программа вычисляет $\sqrt{x}$ один раз при построении ключа, после чего каждый корень стоит одно умножение.

4.4. Неприводимость: тест Бен-Ора

Ключевой факт: над $\text{GF}(q)$

$$x^{q^i} - x = \prod_{h \text{ неприводим, унитарен, } \deg h \mid i} h(x).$$

Корни левой части – ровно элементы $\text{GF}(q^i)$, а неприводимый h степени d имеет корни в $\text{GF}(q^i)$ тогда и только тогда, когда $d \mid i$.

Если g степени t приводим, у него есть неприводимый множитель степени $d \leq t/2$, и он делит $x^{q^d} - x$. Отсюда **тест Бен-Ора**:

$$g \text{ неприводим} \Leftrightarrow \gcd(g(x), x^{q^i} - x \pmod{g}) = 1 \quad \text{для } i = 1, \dots, \lfloor t/2 \rfloor.$$

x^{q^i} никогда не вычисляется целиком: держится $h = x^{q^i} \pmod{g}$, и переход к $i + 1$ – это m возведений h в квадрат по модулю g (ведь $q = 2^m$).

Почему «нет корней» недостаточно

Для степеней 2 и 3 неприводимость равносильна отсутствию корней в поле: разложение обязано содержать линейный множитель. Но уже со степени 4 многочлен может быть произведением двух неприводимых квадратных и не иметь корней. В тестах программы это проверяется явно: тест Бен-Ора отвергает произведение двух неприводимых квадратных многочленов.

Генерация g . Берётся случайный унитарный многочлен степени t и проверяется тестом, пока не пройдёт. Число унитарных неприводимых степени t над $\text{GF}(q)$ задаётся формулой Гаусса

$$N_{q(t)} = \frac{1}{t} \sum_{d|t} \mu(d) q^{t/d} \approx \frac{q^t}{t},$$

то есть случайный кандидат подходит с вероятностью около $1/t$ и нужно в среднем около t попыток. Формула служит и тестом: программа перебирает все унитарные многочлены малых степеней и сверяет количество неприводимых с $N_{q(t)}$, например $N_4(4) = (256 - 16)/4 = 60$.

5. Линейные коды

5.1. Определения

Линейный код $[n, k, d]$ над $\text{GF}(2)$ – подпространство $C \subset \text{GF}(2)^n$ размерности k с минимальным расстоянием d . В нём 2^k кодовых слов, скорость кода $R = k/n$.

- **Порождающая матрица** G ($k \times n$): её строки – базис C , кодирование $c = mG$.
- **Проверочная матрица** H $((n - k) \times n)$: $C = \{c : cH^T = 0\}$, так что $GH^T = 0$.
- **Систематический вид**: $G = [I_k | A^T]$ и $H = [A | I_{n-k}]$ задают один код, так как $GH^T = I_k A^T + A^T I_{n-k} = A^T + A^T = 0$. В этом виде первые k бит кодового слова совпадают с сообщением.

У одного кода много порождающих матриц: SG для любой невырожденной S порождает тот же код, лишь в другом базисе. А GP порождает **эквивалентный** код с переставленными координатами.

5.2. Расстояние и исправление ошибок

Для линейного кода $d(u, v) = w(u - v)$, а $u - v \in C$, поэтому минимальное расстояние равно минимальному весу ненулевого кодового слова. Шары радиуса $\lfloor (d - 1)/2 \rfloor$ вокруг кодовых слов не пересекаются, и код исправляет до $\lfloor (d - 1)/2 \rfloor$ ошибок: ближайшее кодовое слово определяется однозначно.

5.3. Синдром и трудность декодирования

Если принято $y = c + e$, то синдром $yH^T = cH^T + eH^T = eH^T$ зависит только от ошибки. Декодирование – это поиск e малого веса с заданным синдромом. Для **произвольного** линейного кода эта задача NP-трудна (Берлекэмп, Мак-Элис, ван Тилбург, 1978). На этом и держится криптосистема: для кодов Гоппы есть быстрый декодер, но только если известна их структура.

6. Двоичные коды Гоппы

6.1. Определение

Выбираются носитель $L = (\alpha_1, \dots, \alpha_n)$ – n различных элементов $\text{GF}(2^m)$ – и многочлен $g(x)$ степени t над $\text{GF}(2^m)$ с $g(\alpha_i) \neq 0$. Двоичный код Гоппы:

$$\Gamma(L, g) = \{c \in \text{GF}(2)^n : S_c(x) = \sum_{i=1}^n \frac{c_i}{x - \alpha_i} \equiv 0 \pmod{g(x)}\}.$$

Дробь $1/(x - \alpha_i)$ – обратный элемент в $\text{GF}(2^m)[x]/(g)$. Он существует, поскольку $\gcd(x - \alpha_i, g) = 1$ в точности при $g(\alpha_i) \neq 0$. Для неприводимого g степени $t \geq 2$ корней нет вовсе, так что условие выполняется для любого носителя. Код линеен: S_c линейно зависит от c .

Обратный элемент можно выписать явно. Так как $g(x) - g(\alpha)$ делится на $x - \alpha$,

$$\frac{1}{x - \alpha} \equiv -\frac{1}{g(\alpha)} \cdot \frac{g(x) - g(\alpha)}{x - \alpha} \pmod{g}.$$

Проверка: $(x - \alpha) \cdot (-(g(x) - g(\alpha))/((x - \alpha)g(\alpha))) = 1 - g(x)/g(\alpha) \equiv 1$.

6.2. От определения к проверочной матрице

Пусть $g(x) = \sum_{j=0}^t g_j x^j$. Тогда

$$\frac{g(x) - g(\alpha)}{x - \alpha} = \sum_{j=1}^t g_j \sum_{s=0}^{j-1} x^s \alpha^{j-1-s}.$$

Подставим в S_c . Условие $S_c \equiv 0$ – это обращение в ноль многочлена степени меньше t , то есть t линейных условий на c (по одному на коэффициент при x^s). В матричной записи они образуют $H' = CVD$:

$$V = \begin{pmatrix} 1 & 1 & \dots & 1 \\ \alpha_1 & \alpha_2 & \dots & \alpha_n \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_1^{t-1} & \alpha_2^{t-1} & \dots & \alpha_n^{t-1} \end{pmatrix}, \quad D = \text{diag}\left(\frac{1}{g(\alpha_1)}, \dots, \frac{1}{g(\alpha_n)}\right),$$

а C – треугольная матрица из коэффициентов g_j с $g_t = 1$ на диагонали. C обратима, поэтому $H' = CVD$ и $H = VD$ задают **один и тот же** код. Так и строит программа: элемент H в строке j и столбце i равен $\alpha_i^j / g(\alpha_i)$.

6.3. Параметры

Размерность. H имеет t строк над $\text{GF}(2^m)$. Каждый элемент поля – это m бит, и условие $\sum c_i h_{ji} = 0$ в поле распадается на m двоичных условий. Двоичная H получается размера $mt \times n$, откуда

$$k = n - \text{rank } H \geq n - mt.$$

Ранг может оказаться меньше mt , поэтому программа берёт k из результата метода Гаусса, а не из формулы.

Расстояние $d \geq 2t + 1$. Пусть $c \neq 0$ – кодовое слово веса w с носителем E . Положим $f(x) = \prod_{i \in E} (x - \alpha_i)$. Логарифмическая производная даёт

$$f' \frac{x}{f(x)} = \sum_{i \in E} \frac{1}{x - \alpha_i} = S_c(x) \equiv 0 \pmod{g},$$

и так как $\gcd(f, g) = 1$, то $g \mid f'$. В характеристике 2 у производной остаются только чётные степени $((x^{2j+1})' = x^{2j}, (x^{2j})' = 0)$, поэтому $f' = h^2$ для некоторого h . Неприводимый g , деля h^2 , делит и h , откуда $g^2 \mid f'$. При этом $f' \neq 0$: иначе f был бы квадратом и имел бы кратные корни, а все α_i различны. Значит,

$$2t = \deg g^2 \leq \deg f' \leq w - 1 \Rightarrow w \geq 2t + 1.$$

Код исправляет t ошибок. Для **небинарного** кода Гоппы тот же довод даёт лишь $d \geq t + 1$: «удвоение» – заслуга характеристики 2.

6.4. Построение G

Двоичная H приводится методом Гаусса к виду $[A \mid I_r]$, $r = \text{rank } H$. Нулевые строки отбрасываются, и получается $G = [I_k \mid A^T]$.

Ловушка: перестановка столбцов

Если правый $r \times r$ блок H вырожден, опорные элементы приходится брать из других столбцов, и столбцы переставляются. Тогда **ту же** перестановку нужно применить к носителю L , иначе G и декодер будут описывать разные коды, и декодирование будет «иногда» ломаться. Программа ищет опорные элементы сначала в правых r столбцах слева направо, поэтому в типичном случае перестановка тождественна.

7. Декодер Паттерсона

7.1. Постановка

Принято $y = c + e$, $c \in \Gamma(L, g)$, $w(e) \leq t$, E – позиции ошибок. Цель – найти E . Синдром от кодового слова обнуляется:

$$S(x) = S_y(x) = S_e(x) = \sum_{i \in E} \frac{1}{x - \alpha_i} \pmod{g}.$$

7.2. Ключевое уравнение

Введём **многочлен локаторов ошибок**

$$\sigma(x) = \prod_{i \in E} (x - \alpha_i), \quad \deg \sigma = |E| \leq t.$$

Его корни – элементы носителя в позициях ошибок. Как и в разделе 6.3, $\sigma' / \sigma = \sum_{i \in E} 1/(x - \alpha_i)$, поэтому

$$S(x)\sigma(x) \equiv \sigma'(x) \pmod{g(x)}.$$

7.3. Разложение на квадраты

Разобьём σ на чётную и нечётную части (раздел 4.3):

$$\sigma(x) = a(x)^2 + x \cdot b(x)^2, \quad \deg a \leq \lfloor t/2 \rfloor, \quad \deg b \leq \lfloor (t-1)/2 \rfloor.$$

В характеристике 2 $(a^2)' = 2aa' = 0$ и $(xb^2)' = b^2$, значит $\sigma' = b^2$. Подставим в ключевое уравнение и умножим на $T = S^{-1} \pmod{g}$ (он существует: $S \neq 0$, а по модулю неприводимого g обратим любой ненулевой остаток):

$$S(a^2 + xb^2) \equiv b^2 \Rightarrow a^2 + xb^2 \equiv Tb^2 \Rightarrow a^2 \equiv (T+x)b^2.$$

Корень по модулю g существует и единствен (раздел 4.3), а извлечение корня аддитивно, поэтому

$$a(x) \equiv b(x) \cdot \tau(x) \pmod{g(x)}, \quad \tau = \sqrt{T+x} \pmod{g}.$$

Уравнение второй степени свелось к линейному.

7.4. Решение укороченным алгоритмом Евклида

Нужна пара (a, b) с $a \equiv b\tau$ и ограничениями на степени из раздела 7.3. Запустим расширенный алгоритм Евклида на (g, τ) : по инварианту из раздела 4.1 каждый остаток удовлетворяет $r_i \equiv v_i \tau \pmod{g}$. Остановимся на **первом** остатке степени не выше $\lfloor t/2 \rfloor$ и возьмём $a = r_i$, $b = v_i$. Тогда предыдущий остаток имеет степень не меньше $\lfloor t/2 \rfloor + 1$ и

$$\deg b = t - \deg r_{i-1} \leq t - \lfloor t/2 \rfloor - 1 = \lfloor (t-1)/2 \rfloor.$$

Обе оценки выполнены, и $\deg \sigma = \deg(a^2 + xb^2) \leq t$.

Единственность. Пусть (a, b) и (a_1, b_1) – два решения с этими оценками. Тогда $ab_1 - a_1b \equiv \tau bb_1 - \tau bb_1 = 0$ по модулю g , а степень левой части меньше t , так что $ab_1 = a_1b$ точно. У истинного решения $\gcd(a, b) = 1$: общий множитель вошёл бы в σ в квадрате, а корни σ простые. Значит, найденная пара отличается от истинной лишь скалярным множителем, и корни σ те же. В сквозном примере так и получается: $\sigma = \alpha^3(x + \alpha^3)(x + \alpha^{10})$.

Особый случай $T = x$

Если $S^{-1} = x$, то $\tau = 0$, Евклид останавливается сразу с $a = 0$, $b = 1$, и $\sigma = x$. Это одна ошибка в позиции, где $\alpha_i = 0$: $S = 1/x$. Отдельная ветка в коде не нужна, но тест на этот случай есть.

7.5. Поиск корней и проверки

Позиции ошибок – те i , для которых $\sigma(\alpha_i) = 0$. Корни ищутся подстановкой всех элементов носителя (схема Горнера), и биты y в найденных позициях инвертируются. Если ошибок больше t , алгоритм всё равно выдаст **какой-то** σ , поэтому результат проверяется:

- число найденных корней должно равняться $\deg \sigma$ (у верного σ все корни простые и лежат в носителе);
- исправленный вектор должен иметь нулевой синдром.

Иначе блок объявляется искажённым. Остаётся случай, когда при $t+1$ и более ошибках y оказался в пределах t от **другого** кодового слова: тогда декодер вернёт его, и обнаружить подмену невозможно в принципе. Исходное слово при этом не возвращается никогда, что проверено тестом.

Сложность. Синдром – $O(nt)$ операций в поле (обратные к $x - \alpha_i$ вычисляются заранее), обращение, корень и Евклид – $O(t^2)$, поиск корней – $O(nt)$.

8. Криптосистема

8.1. Генерация ключей

1. Выбрать случайный носитель L (n различных элементов $\text{GF}(2^m)$) и случайный неприводимый g степени t (раздел 4.4).
2. Построить H , привести к систематическому виду, получить $G = [I_k \mid A^T]$ и согласованно переставить L (раздел 6.4).
3. Выбрать случайную невырожденную S ($k \times k$) и вычислить S^{-1} .
4. Выбрать случайную перестановку P (Фишер – Йетс) и P^{-1} .
5. Открытый ключ: $(G' = SG P, t)$. Закрытый: (L, g, S^{-1}, P^{-1}) .

Невырожденная S ищется перебором случайных матриц. Доля обратимых матриц $k \times k$ над $\text{GF}(2)$ равна

$$\prod_{i=1}^k (1 - 2^{-i}) \rightarrow 0.2888\dots,$$

поэтому в среднем хватает 3–4 попыток. Каждая попытка – метод Гаусса – Жордана над $[S \mid I]$, который заодно даёт S^{-1} .

8.2. Зашифрование и расшифрование

$$\text{зашифрование: } c = mG' + e, \quad w(e) = t.$$

$$\text{расшифрование: } y = cP^{-1}, \quad mSG = \text{Decode}(y), \quad mS = \text{первые } k \text{ бит}, \quad m = (mS)S^{-1}.$$

8.3. Корректность

Раскроем y :

$$cP^{-1} = mSGPP^{-1} + eP^{-1} = (mS)G + eP^{-1}.$$

- $(mS)G$ – кодовое слово кода Гоппы: mS – просто другой вектор длины k .
- eP^{-1} – переставленный вектор ошибок, его вес по-прежнему t : перестановка не меняет числа единиц.

Декодер Паттерсона, зная L и g , исправляет t ошибок и возвращает $(mS)G$. Из-за систематического вида $G = [I_k \mid A^T]$ первые k бит этого слова равны mS , и умножение на S^{-1} даёт m .

8.4. Зачем нужны S и P

- **Без S** матрица $G' = GP$ сохраняла бы систематическую структуру с точностью до перестановки столбцов. Первые k «информационных» позиций шифротекста были бы сообщением плюс лишь часть ошибок, то есть открытый текст читался бы почти напрямую. S перемешивает базис кода.
- **Без P** порядок координат совпадал бы с порядком носителя, и структура кода Гоппы была бы видна из G' . P превращает код в эквивалентный с переставленными координатами.

Вместе они делают G' похожей на порождающую матрицу **случайного** линейного кода, для которого эффективного декодера нет.

Вероятностное шифрование

Вектор ошибок выбирается заново при каждом зашифровании, поэтому одно и то же сообщение даёт разные шифротексты. Число возможных $e - \binom{n}{t}$; для $n = 1024, t = 50$ это около 2^{284} .

9. Кодирование сообщений

Криптосистема шифрует блоки по k бит, а сообщение – это байты.

1. **Порядок бит.** Бит $8j + b$ потока – бит b байта j , младший бит первым. Этому же порядка придерживаются упаковка векторов в `u64` и запись шифроблоков.
2. **Дополнение.** К битам сообщения дописывается одна единица и нули до длины, кратной k (побитовый вариант ISO/IEC 7816-4). Дополнение есть **всегда**, поэтому число блоков равно $\lceil 8|M| / k \rceil + 1$.
3. **Шифроблок.** Каждый блок из n бит записывается в $\lceil n/8 \rceil$ байт; лишние биты последнего байта нулевые и проверяются при расшифровании.

4. **Снятие дополнения.** После расшифрования ищется последняя единица: она должна стоять на границе байта и в пределах последних k бит. Всё, начиная с неё, отбрасывается.

Шифротекст длиннее открытого текста примерно в n/k раз:

Набор	m	n	t	k	Размер G' и расширение
toy	4	16	2	8	16 байт; $n/k = 2$, 1 байт текста $\rightarrow$ 2 блока, 4 байта
small	8	256	8	192	6144 байта; $n/k \approx 1.33$
original (1978)	10	1024	50	524	67 072 байта; $n/k \approx 1.95$

10. Стойкость

10.1. Две атаки

Противник знает G' , t и c . У него два пути.

1. **Структурная атака:** восстановить по G' код Гоппы, то есть L и g . Для двоичных кодов Гоппы за 45 с лишним лет такой атаки не найдено; известен лишь различитель для кодов с очень высокой скоростью $k/n \rightarrow 1$, которые в схеме не используются. Именно поэтому нельзя брать другие алгебраические коды: для кодов Рида – Соломона (схема Нидеррайтера), Рида – Маллера и ряда других структура восстанавливается.
2. **Декодирование общего вида:** найти e веса t по c , как для случайного кода. Лучшие методы – декодирование по информационным множествам (ISD).

10.2. Декодирование по информационным множествам

Идея Прейнджа (1962): выбрать k случайных позиций и надеяться, что в них нет ошибок. Если подматрица G' на этих столбцах обратима, то m восстанавливается решением линейной системы, после чего проверяется, что $c - mG'$ имеет вес t . Вероятность удачи одной попытки

$$p = \frac{\binom{n-t}{k}}{\binom{n}{k}},$$

и стоимость около k^3/p операций:

Параметры	$\log_2 p$	$\log_2(k^3/p)$
$n = 16, k = 8, t = 2$	-2.1	11
$n = 256, k = 192, t = 8$	-16.5	39
$n = 1024, k = 524, t = 50$	-53.6	81
$n = 3488, k = 2720, t = 64$	-142.8	177

Это грубая оценка базового метода. Улучшения (Штерн, 1989, и последующие) снижают показатель. Параметры 1978 года были практически взломаны в 2008 году (Бернштейн, Ланге, Петерс), примерно за 2^{60} операций. Учебные наборы **toy** и **small** стойкости не имеют и нужны только для наглядности.

10.3. Слабости базовой схемы

Схема «как в учебнике» (она и реализована) детерминирована относительно m при фиксированном e и податлива.

- **Податливость.** $c + g'_j$, где g'_j – строка G' , – корректный шифротекст сообщения $m + u_j$. Шифротекст можно предсказуемо менять, не зная ключа.
- **Повторная отправка.** Если одно и то же m зашифровано дважды, то $c_1 + c_2 = e_1 + e_2$. Вес этой суммы около $2t$, и она раскрывает большую часть позиций ошибок. Остаток находится перебором, и m восстанавливается без ключа.
- **Атака с подобранным шифротекстом.** Если переворачивать отдельные биты c и смотреть, удаётся ли расшифрование, то позиции ошибок видны напрямую: перевёрнутый бит в позиции ошибки даёт

$t - 1$ ошибку и успешное расшифрование, в любой другой позиции $-t + 1$ ошибку и отказ. Этот эффект есть в тесте `one_flipped_bit_is_fatal_or_absorbed` и на рисунке 2.6 отчёта.

Практические варианты это исправляют: конверсия Кобары – Имаи для шифрования, а стандартный кандидат **Classic McEliece** использует схему КЕМ-инкапсуляции ключа (обычно в дуальной форме Нидеррайтера), где сообщение – это сам случайный e , а ключ – хэш от него.

10.4. Современные параметры

Набор Classic McEliece	m	n	t	k	Открытый ключ
<code>mceliece348864</code>	12	3488	64	2720	261 120 байт
<code>mceliece6960119</code>	13	6960	119	5413	1 047 319 байт

Главная плата за стойкость – размер открытого ключа: сотни килобайт против сотен байт у RSA или решёточных схем. Зато зашифрование – это одно умножение вектора на матрицу, а стойкость к квантовым атакам не опирается на задачи теории чисел: алгоритм Шора здесь неприменим.

11. Сквозной пример на $GF(16)$

Все значения ниже посчитаны независимо от программы и сверены с ней.

Поле и многочлен. $GF(16)$ с $f = z^4 + z + 1$ (таблица в разделе 3.3), $t = 2$,

$$g(x) = x^2 + x + \alpha^7.$$

Квадратный многочлен неприводим тогда и только тогда, когда у него нет корней. Подстановка всех 16 элементов это подтверждает. По модулю g имеем $x^2 \equiv x + \alpha^7$ и $\sqrt{x} \equiv x + \alpha^{11}$ (проверка: $(x + \alpha^{11})^2 = x^2 + \alpha^{22} = x + \alpha^7 + \alpha^7 = x$).

Слагаемые синдрома. Для позиций с $\alpha_i = \alpha^3$ и $\alpha_i = \alpha^{10}$:

$$\frac{1}{x + \alpha^3} \equiv \alpha^3 x + \alpha^2, \quad \frac{1}{x + \alpha^{10}} \equiv \alpha^6 x + \alpha^{11}.$$

Проверим первое: $(x + \alpha^3)(\alpha^3 x + \alpha^2) = \alpha^3 x^2 + (\alpha^2 + \alpha^6)x + \alpha^5$. С учётом $x^2 \equiv x + \alpha^7$ получаем $(\alpha^3 + \alpha^2 + \alpha^6)x + \alpha^{10} + \alpha^5 = 0 \cdot x + 1$, так как $\alpha^2 + \alpha^3 = \alpha^6$ и $\alpha^5 + \alpha^{10} = 1$.

11.1. Одна ошибка (в позиции α^3)

Шаг	Значение
S	$\alpha^3 x + \alpha^2$
$T = S^{-1}$	$x + \alpha^3$ (обратное к обратному)
$T + x$	α^3
$\tau = \sqrt{\alpha^3}$	α^9 , так как $\alpha^{18} = \alpha^3$
Евклид	$\deg \tau = 0 \leq 1$: сразу $a = \alpha^9$, $b = 1$
$\sigma = a^2 + xb^2$	$\alpha^{18} + x = x + \alpha^3$
Корень	α^3 – найдена ошибка

11.2. Две ошибки (в позициях α^3 и α^{10})

Шаг	Значение
S	$(\alpha^3 + \alpha^6)x + (\alpha^2 + \alpha^{11}) = \alpha^2 x + \alpha^9$
$T = S^{-1}$	$\alpha^{14}x + \alpha^8$
$T + x$	$\alpha^3 x + \alpha^8$
$\tau = \sqrt{T + x}$	$p_0 = \sqrt{\alpha^8} = \alpha^4, p_1 = \sqrt{\alpha^3} = \alpha^9; \tau = \alpha^4 + \alpha^9(x + \alpha^{11}) = \alpha^9 x + \alpha^8$

Евклид	$\deg \tau = 1 \leq 1$: сразу $a = \alpha^9 x + \alpha^8$, $b = 1$
$\sigma = a^2 + xb^2$	$\alpha^{18}x^2 + \alpha^{16} + x = \alpha^3x^2 + x + \alpha$
Корни	α^3 и α^{10} – обе ошибки найдены

Действительно, $\alpha^3(x + \alpha^3)(x + \alpha^{10}) = \alpha^3x^2 + \alpha^3(\alpha^3 + \alpha^{10})x + \alpha^{16} = \alpha^3x^2 + x + \alpha$, так как $\alpha^3 + \alpha^{10} = \alpha^{12}$ и $\alpha^3 \cdot \alpha^{12} = 1$. Найденный σ отличается от $\prod(x - \alpha_i)$ множителем α^3 – ровно как обещает раздел 7.4.

Почему здесь нет шагов Евклида

При $t = 2$ граница остановки $\lfloor t/2 \rfloor = 1$, а $\deg \tau \leq t - 1 = 1$ всегда, поэтому алгоритм останавливается на нулевом шаге и $b = 1$. Для $t = 50$ до остатка степени 25 проходит порядка 25 шагов деления.

12. Реализация

12.1. Где что лежит

Функция	Математика
<code>Field::with_modulus</code>	таблицы α^i и проверка примитивности (3.3)
<code>Field::sqrt</code>	$a^{2^{m-1}}$ через логарифм (3.4)
<code>Poly::div_rem</code> , <code>gcd</code> , <code>inv_mod</code>	деление, НОД, обращение по модулю (4.1)
<code>Poly::sqrt_x_mod</code> , <code>sqrt_mod</code>	$\sqrt{x} = x^{2^{m-1}}$, $\sqrt{p} = p_0 + \sqrt{x}p_1$ (4.3)
<code>Poly::is_irreducible</code>	тест Бен-Ора (4.4)
<code>Poly::partial_ext_gcd</code>	укороченный Евклид с инвариантом $r \equiv v\tau$ (7.4)
<code>Matrix::systematic</code>	приведение H к $[A \mid I]$ с перестановкой столбцов (6.4)
<code>Matrix::inverse</code> , <code>random_invertible</code>	Гаусс – Жордан над $[S \mid I]$ (8.1)
<code>Goppa::from_parts</code>	$H = VD$, $G = [I \mid A^T]$, перестановка L , предвычисления (6.2, 6.4)
<code>Goppa::syndrome</code>	$S(x) = \sum y_i/(x - \alpha_i)$ (7.1)
<code>Goppa::error_locator</code> , <code>decode</code>	Паттерсон: T , τ , (a, b) , σ , корни, проверки (7.2–7.5)
<code>McEliece::keygen_with_rng</code>	$G' = SGP$ (8.1)
<code>McEliece::encrypt_block</code> , <code>decrypt_block</code>	$c = mG' + e$ и обратный путь (8.2)
<code>McEliece::encrypt</code> , <code>decrypt</code>	дополнение и формат шифроблоков (9)

12.2. Что проверяют тесты

- Умножение по таблицам полностью совпадает с прямым умножением многочленов для $\text{GF}(2^4)$ и $\text{GF}(2^8)$; $aa^{-1} = 1$ и $(\sqrt{a})^2 = a$ для всех a .
- Тест Бен-Ора совпадает с формулой Гаусса $N_{q(t)}$ для $\text{GF}(4)$ (степени 1–4) и $\text{GF}(16)$ (степень 2).
- $g(x) = x^2 + x + \alpha^7$ неприводим над $\text{GF}(16)$, а $x^2 + x + \alpha$ приводим.
- $GH^T = 0$, и синдром каждой строки G нулевой.
- Для кода $\text{GF}(16)$, $n = 16$, $t = 2$ перебором всех кодовых слов подтверждено $d \geq 5 = 2t + 1$.
- Декодер исправляет от 0 до t случайных ошибок для пяти наборов параметров и для $n = 1024$, $t = 50$; случай $T = x$ проверен отдельно.
- При $t + 1$ ошибках декодер либо сообщает об ошибке, либо возвращает **другое** кодовое слово на расстоянии не больше t , но никогда исходное.

13. Вопросы на защите

? На какой трудной задаче основана криптосистема?

На декодировании произвольного линейного кода: по вектору c и порождающей матрице найти кодовое слово на расстоянии t , то есть вектор ошибок веса t . Задача NP-трудна в общем случае (1978). Для кода Гоппы есть быстрый декодер, но он требует знания L и g , а G' скрывает их, выглядя как матрица случайного кода.

? Почему поле $\text{GF}(2^m)$ – это не числа по модулю 2^m ?

В $\mathbb{Z}/2^m$ есть делители нуля ($2 \cdot 2^{m-1} = 0$), и у чётных чисел нет обратных, так что это не поле. $\text{GF}(2^m)$ строится как многочлены по модулю неприводимого многочлена степени m . Размер тот же, 2^m элементов, но умножение другое.

? Зачем многочлен поля должен быть именно примитивным?

Для корректности поля хватило бы неприводимости. Примитивность нужна для таблиц: корень α должен порождать все $2^m - 1$ ненулевых элементов, чтобы каждый записывался как α^i и умножение сводилось к сложению показателей. Пример неприводимого, но не примитивного многочлена – $z^4 + z^3 + z^2 + z + 1$: его корень имеет порядок 5.

? Почему квадратный корень в $\text{GF}(2^m)$ всегда существует?

В характеристике 2 возведение в квадрат аддитивно, $(a + b)^2 = a^2 + b^2$, и является автоморфизмом конечного поля (Фробениус). Биекция поля на себя означает, что у каждого элемента ровно один прообраз – корень. Формула: $\sqrt{a} = a^{2^{m-1}}$, так как $a^{2^m} = a$.

? Зачем многочлен Гоппы должен быть неприводимым?

По трём причинам. Чтобы $g(\alpha_i) \neq 0$ для любого носителя и все $1/(x - \alpha_i)$ существовали. Чтобы $\text{GF}(2^m)[x]/(g)$ было полем и в Паттерсоне существовали S^{-1} и $\sqrt{T + x}$. И чтобы в доказательстве $d \geq 2t + 1$ из $g \mid h^2$ следовало $g \mid h$.

? Почему проверки «нет корней» недостаточно для неприводимости?

Она достаточна только для степеней 2 и 3. Начиная со степени 4 многочлен может раскладываться на неприводимые множители степени 2 и выше, не имея корней. Поэтому используется тест Бен-Ора: $\gcd(g, x^{q^i} - x) = 1$ для $i \leq t/2$ исключает множители всех степеней до $t/2$.

? Откуда $k \geq n - mt$?

Проверочная матрица над $\text{GF}(2^m)$ имеет t строк. Каждое условие над полем – это m двоичных условий, так что двоичная H имеет mt строк и ранг не больше mt . Размерность кода $k = n - \text{rank } H \geq n - mt$. Равенство бывает не всегда, поэтому k берётся из метода Гаусса.

? Почему двоичный код Гоппы исправляет t ошибок, а не $t/2$?

Для кодового слова с носителем E и $f = \prod_{i \in E} (x - \alpha_i)$ условие кода означает $g \mid f'$. В характеристике 2 производная – полный квадрат, $f' = h^2$, и из неприводимости g следует $g^2 \mid f'$. Отсюда $2t \leq \deg f' \leq w - 1$, то есть $d \geq 2t + 1$. Для небинарных кодов этого «удвоения» нет и гарантируется лишь $d \geq t + 1$.

? Выведите ключевое уравнение Паттерсона.

Синдром равен $S = \sum_{i \in E} 1/(x - \alpha_i) = \sigma'/\sigma$ – это логарифмическая производная многочлена локаторов. Отсюда $S\sigma \equiv \sigma' \pmod{g}$. Затем $\sigma = a^2 + xb^2$ даёт $\sigma' = b^2$; умножение на $T = S^{-1}$ даёт $a^2 \equiv (T + x)b^2$, и после извлечения корня $a \equiv \tau b$, $\tau = \sqrt{T + x}$.

? Как из $a \equiv \tau b$ найти a и b ?

Расширенным алгоритмом Евклида на (g, τ) с остановкой на первом остатке степени не выше $\lfloor t/2 \rfloor$. Инвариант $r_i \equiv v_i \tau$ даёт $a = r_i$, $b = v_i$, а $\deg v_i = t - \deg r_{i-1} \leq \lfloor (t - 1)/2 \rfloor$. Решение с такими степенями единственно с точностью до множителя, поэтому корни $\sigma = a^2 + xb^2$ – ровно позиции ошибок.

? Что делает декодер, если ошибок больше t ?

Паттерсон всё равно выдаст какой-то σ . Программа проверяет, что число его корней на носителе равно $\deg \sigma$ и что после исправления синдром нулевой; иначе блок объявляется искажённым. Если же y случайно попал в шар радиуса t вокруг **другого** кодового слова, декодер вернёт его: это принципиально необнаружимо, но исходное слово не вернётся никогда.

? Докажите корректность расшифрования.

$cP^{-1} = (mS)G + eP^{-1}$. Первое слагаемое – кодовое слово, второе – вектор веса t , потому что перестав

новка сохраняет вес. Декодер возвращает $(mS)G$. Благодаря виду $G = [I_k \mid A^T]$ его первые k бит – это mS , и $m = (mS)S^{-1}$.

? Зачем нужны S и P по отдельности?

S меняет базис кода и прячет систематический вид: без неё сообщение читалось бы в «информационных» позициях шифротекста. P переставляет координаты и скрывает связь позиций с носителем L . Вместе они делают G' неотличимой от порождающей матрицы случайного кода.

? Почему нельзя взять вместо кода Гоппы код Рида – Соломона?

Для кодов Рида – Соломона и многих других алгебраических кодов найдены структурные атаки: по G' восстанавливается сам код (атака Сидельникова – Шестакова на вариант Нидеррайтера). Для двоичных кодов Гоппы такой атаки нет. Их много (порядка $2^{mt}/t$ многочленов g плюс выбор носителя), и их структура по G' не видна.

? Почему шифрование вероятностное и чем это полезно?

Вектор ошибок e каждый раз новый и случайный; вариантов $\binom{n}{t}$, для параметров 1978 года около 2^{284} . Одинаковые сообщения дают разные шифротексты, и сравнение шифротекстов ничего не выдаёт.

? Какие слабости у базовой схемы?

Податливость: прибавив к c строку G' , получаем шифротекст другого сообщения. Повторная отправка: $c_1 + c_2 = e_1 + e_2$ раскрывает позиции ошибок. Атака с подобранным шифротекстом: переворот бита в позиции ошибки не мешает расшифрованию, в любой другой позиции – мешает, и так вычисляется e . Лечится конверсиями (Кобара – Имаи) или схемой KEM, как в Classic McEliece.

? Насколько стойки параметры 1978 года?

Базовая оценка ISD по Преинджу – около 2^{81} операций, улучшенные алгоритмы дают меньше. В 2008 году их практически взломали примерно за 2^{60} операций (Бернштейн, Ланге, Петерс). Современный минимальный набор Classic McEliece: $n = 3488$, $t = 64$, открытый ключ 261 120 байт.

? Главный недостаток криптосистемы?

Размер открытого ключа: $k \times n$ бит, то есть 67 072 байта для параметров 1978 года и сотни килобайт для современных – на порядки больше, чем у RSA. Вторичный недостаток – шифротекст длиннее текста в n/k раз. Зато шифрование и расшифрование быстрые, а стойкость не зависит от задач, которые решает квантовый компьютер.

? Почему в примере с $t = 2$ алгоритм Евклида не делает ни одного шага?

Остановка наступает на остатке степени не выше $\lfloor t/2 \rfloor = 1$, а τ как остаток по модулю g степени 2 всегда имеет степень не выше 1. Поэтому сразу $a = \tau$, $b = 1$. Реальная работа алгоритма видна при больших t : для $t = 50$ проходит порядка 25 делений.